The Readers–Writers Problem, Solved Four Ways: semaphore, monitor, Dekker, Peterson

What is proved about every interleaving, what is measured on real threads, and where the two disagree

Title The readers-writers problem, solved four ways
Subtitle Semaphore, monitor, Dekker and Peterson, proved and measured
Course Advanced Operating Systems
Exercise 7 — readers and writers
Method exhaustive model checking plus a threaded benchmark harness
Solutions modelled 17 protocols, 2 to 3 processes each
Solutions benchmarked 15 locks on real threads
Deliverable Persian and English papers (DOCX and PDF), figures, code and tests
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-7-Readers-Writers-Comparison

Table of contents

3 Abstract and keywords
4 1. The problem, and why four answers exist
5 2. Two instruments: a proof and a measurement
6 2.2 The threaded harness
7 2.3 The bug, and the fix
8 3. The solutions, in the form the checker explores
9 3.2 The monitor
10 3.3 The two classic algorithms
11 3.4 Peterson
12 4. What is proved
14 4.1 Reading the table
15 4.2 How big each proof was
16 4.3 What a counterexample looks like
17 4.4 Starvation, found as a fair cycle
18 5. Dekker and Peterson: correct, fast, and only two of you
19 6. What the locks cost
20 6.1 What the no-barging queue costs
21 7. The property the problem exists for
22 8. Starvation, measured
23 8. Starvation, the numbers
24 8.1 Reading the starvation figures
25 8.2 The mirror image, and where measurement gives up
26 9. Does adding readers help?
27 10. Which one to use
28 11. Method, artefacts and limits

Abstract and keywords

Abstract

The readers-writers problem asks for a lock that keeps writers exclusive of everybody while letting any number of readers share the section. Four classic mechanisms are usually offered as the answer, and they are not interchangeable: a counting semaphore, a monitor with condition variables, and the two classic two-process software solutions of Dekker and Peterson. This exercise builds all of them, then asks two separate questions of each. The first is proved: an exhaustive model checker walks every reachable interleaving of a two- or three-process protocol and returns a verdict on safety, deadlock freedom, starvation freedom and, the property the problem actually exists for, whether two readers can be inside together. The second is measured: a threaded harness runs each lock under load and checks the invariant on every single entry.

The two answers do not always point the same way, and the disagreements are the interesting part. Readers preference is safe, deadlock free and fast, and it starves the writer completely: in 6 of 9 half-second trials with six readers and one writer the writer took under one per cent of the traffic, and in the rest it took almost all of it — the outcome depends on who wins the opening race and then locks in. A Mesa monitor is safe and admits concurrent reads, but because a notified thread has to re-compete for the monitor lock it does not bound waiting: the checker finds a fair cycle in which every participant starves. Hand the woken thread a place in the entry queue instead and the verdict flips. Dekker and Peterson are correct and starvation free as locks, and they are written for exactly two participants, which is the whole of their limitation: they can carry the readers-writers pattern between two readers, and they cannot carry it to a third.

Keywords

readers-writers problem, mutual exclusion, semaphore, monitor, Mesa semantics, Dekker's algorithm, Peterson's algorithm, starvation, model checking, fairness

> **Note:** Every figure in this paper is produced by `exercise-7/code/run_experiments.py` and read from `output/experiments.json` at build time. No number is typed by hand, so the paper cannot drift away from the code that produced it.

1. The problem, and why four answers exist

What has to hold

A readers-writers lock guards one shared object. Any number of readers may hold it at once; a writer must hold it alone. Three properties follow, and the third is the one that makes the problem worth an exercise:

- **Safety** — never two writers inside at once, and never a writer inside together with anybody else.
- **Liveness** — a process that asks eventually gets in. This splits into deadlock freedom (somebody can always still get in) and starvation freedom (everybody eventually does).
- **Concurrency** — two readers really can be inside at once. A plain mutex satisfies the first two properties perfectly and still fails this one, which is the entire reason the problem exists.

Why the four mechanisms are not interchangeable

A semaphore is a counter with a queue. A monitor is a lock plus condition variables, and when a thread waits it gives the lock up. Dekker's and Peterson's algorithms are pure software solutions to mutual exclusion, written with nothing but shared flags, and both are written for exactly two processes. They differ in what they can express, in what they cost, and in what they guarantee about waiting — which is what the rest of this paper measures.

> **Note:** The four mechanisms are compared at two levels throughout: as the *count lock* inside a readers-writers solution, and as the whole solution. Dekker and Peterson can do the first for two participants and simply cannot do the second, and that distinction is proved in section 5 rather than asserted.

2. Two instruments: a proof and a measurement

2.1 The model checker

`code/rw_model.py` enumerates the reachable state space of a protocol. Memory is sequentially consistent, every load and store of a single word is atomic, and a global state is the tuple (shared counters, semaphores, program counters, private registers). A process that cannot proceed contributes a self-loop, which the explorer prunes, so a blocked process simply has no successor and a deadlock shows up as a state from which no entry is reachable any more.

- **Safety** is checked on every state as it is reached.
- **Deadlock freedom** is reverse reachability: is there a state in which somebody is trying, nobody is inside, and no entry is reachable?
- **Starvation freedom** is a fair-cycle search, explained below.
- **Concurrent reads** is the positive property: does some reachable state have two readers inside at once?

Fairness is the part that is easy to get wrong

A cycle in which the victim simply is never scheduled proves nothing: that is an unfair scheduler, not a broken lock. Three separate mistakes were made and caught while building this checker, and each one is worth naming because each produced a confident wrong answer:

- Fairness only for the **victim** lets a third process sit on an enabled step it never takes, hold a lock for ever, and make the victim look starved. Fairness has to be required of every process.
- Pruning the victim's own step into its remainder section removes exactly the transition that proves it was scheduled.
- A **path** that overtakes the victim twice proves nothing; only a closed cycle can be repeated for ever.

> **Note:** The rule the checker finally implements: a cycle is fair for a process when, at every state of the cycle where that process has any enabled step, one of those steps is part of the cycle. If the process is blocked anywhere on the cycle the obligation is empty, and the cycle is genuine starvation.

2.2 The threaded harness

What it runs

`code/rw_sync.py` implements every solution as a real lock and runs it under load. Two design decisions are not incidental.

- **The semaphore is strictly FIFO.** A thread takes a ticket on arrival and is served in ticket order, so a newcomer cannot overtake a waiter. `threading.Semaphore` does not behave that way, and using it would quietly invalidate the fairness proofs below. The paper measures both, so the price of the queue is a number rather than a claim.
- **The invariant is checked on every entry, not at the end.** A lock that lets a writer in beside a reader for a microsecond leaves no trace in any counter read afterwards. The harness records the violation while it is happening, and it records how many readers were ever inside at once.

What the harness caught

Building `wrt` acquisition outside the count mutex — the obvious way to write it, and the way it was written first — opens a window in which a second reader has already raised the count but the first has not yet taken the writer lock. The harness found it in under a second of load. The model had it right all along, which is the argument for having both instruments.

> **Note:** Neither instrument is redundant. The model checker explores two or three processes for a handful of steps and proves statements about all of them; it says nothing about fifteen locks running for a second of wall-clock time. The harness says nothing about the interleaving it happened not to take. Together they caught a bug that each alone would have missed.

2.3 The bug, and the fix

```
# wrong: the count is raised, the mutex is dropped, and only
# then is wrt taken -- in that window a writer can be inside
# together with a reader
with self._mutex:
    self._readers += 1
    first = self._readers == 1
if first:
    self._wrt.acquire()          # too late

# right: wrt is taken while the count is still protected
with self._mutex:
    self._readers += 1
    if self._readers == 1:
        self._wrt.acquire()
```

Readers preference, before and after

Why the window exists

The invariant is not “the count is right”, it is “the writer semaphore is held for exactly as long as any reader is inside”. Between releasing the count mutex and taking `wrt`, the count already says a reader is coming but nothing yet stops a writer. A second reader arriving in that window sees a count of two and walks straight into the section without touching `wrt` at all, so the writer lock no longer means what the protocol assumes it means.

> **Note:** The equivalent mistake on the release side is symmetric and just as easy to make: giving `wrt` back after dropping the count mutex lets a writer in while the count is still above zero.

3. The solutions, in the form the checker explores

Each listing below is the actual modelled program, one line per atomic step, disassembled from `code/rw_model.py` at build time. The interleaving of these steps is exactly what the verdicts in section 4 are about, so the level of detail is the point: a tidy pseudocode version would hide the very steps across which the solutions differ.

3.1 Readers preference

```
 0  p(mutex)
 1  rd = rd + 1
 2  r0 = rd
 3  r0 = r0 - 1
 4  if r0 != 0 goto 6
 5  p(wrt)
 6  v(mutex)
 7  enter
 8  exit
 9  p(mutex)
10  rd = rd - 1
11  r0 = rd
12  if r0 != 0 goto 14
13  v(wrt)
14  v(mutex)
15  remainder; goto 0
```

The reader program of `rw_readers_pref`

`mutex` protects the count and `wrt` excludes writers. The first reader takes `wrt`, the last one gives it back, and everybody in between just walks in. Note that `wrt` is taken at step 5, while `mutex` is still held — the detail section 2.3 is about.

3.2 The monitor

```
 0  p(m)
 1  r0 = wactive
 2  if r0 == 0 goto 4
 3  wait(cv) -> 1
 4  r0 = rd
 5  if r0 == 0 goto 7
 6  wait(cv) -> 1
 7  wactive = 1
 8  v(m)
 9  enter
10  exit
11  p(m)
12  wactive = 0
13  notifyAll(cv)
14  v(m)
15  remainder; goto 0
```

The writer program of `rw_monitor_mesa`

One lock, one condition variable. A writer waits while another writer is active and waits again while a read phase is open; on the way out it clears the flag and wakes everybody. Every wait jumps back to the re-test at step 1 rather than to the top, which is what Mesa semantics require — a notified thread has to look at the condition again, because the notification carried no promise.

And the fair variant

```
 0  p(m)
 1  r0 = wactive
 2  if r0 == 0 goto 4
 3  wait-queued(cv) -> 1
 4  r0 = rd
 5  if r0 == 0 goto 7
 6  wait-queued(cv) -> 1
 7  wactive = 1
 8  v(m)
 9  enter
10  exit
11  p(m)
12  wactive = 0
13  notifyAll+handoff(cv)
14  v(m)
15  remainder; goto 0
```

The same program with `wait-queued` and `notifyAll+handoff`

Two instructions differ. `wait-queued` puts the woken thread into the monitor's entry queue instead of leaving it to race, and `notifyAll+handoff` performs the hand-off atomically at signal time rather than leaving a notified thread waiting to be scheduled. That is the whole of the difference between “every participant can starve” and “only the writer can”.

3.3 The two classic algorithms

```
 0  flag[me] = 1
 1  r0 = flag[other]
 2  if r0 == 0 goto 11
 3  r0 = turn
 4  if r0 == me goto 1
 5  flag[me] = 0
 6  r0 = turn
 7  if r0 == me goto 9
 8  goto 6
 9  flag[me] = 1
10  goto 1
11  enter
12  exit
13  turn = other
14  flag[me] = 0
15  remainder; goto 0
```

Dekker's algorithm, as modelled

Two flags and a turn. Dekker yields: when the other side is interested and it is not my turn, I lower my flag, wait for the turn to come back, and raise it again. The yield is the part that took a correction to get right — the first version jumped into the section on seeing its own turn, which is the opposite of what Dekker does, and the checker duly reported a safety violation.

3.4 Peterson

```
 0  flag[me] = 1
 1  turn = other
 2  r0 = flag[other]
 3  if r0 == 0 goto 7
 4  r0 = turn
 5  if r0 == me goto 7
 6  goto 2
 7  enter
 8  exit
 9  flag[me] = 0
10  remainder; goto 0
```

Peterson's algorithm, as modelled

Peterson removes the busy loop: the turn does the yielding, so the entry protocol is four instructions instead of eleven. Both need two instruction forms the other solutions do not — one to write my own flag and one to read the other's — because a process has to know which of the two it is.

Fewer instructions, fewer places to be interrupted: Dekker explores 443 states and Peterson 267. Neither admits two readers at once, because neither is trying to.

4. What is proved

Every row below is exhaustive: no verdict rests on a truncated state space. The broken solution is the single exception and it is an exception by design — it stops at its first witness, which is a proof that the invariant fails, not a survey of what happens after.

solution	roles	states	safety	deadlock-free	starves	concurrent reads
`semaphore_mutex`	RW	28	holds	yes	none	**no**
`monitor_mesa_mutex`	RW	146	holds	yes	**all of them**	**no**
`monitor_fifo_mutex`	RW	122	holds	yes	none	**no**
`dekker_mutex`	RW	443	holds	yes	none	**no**
`peterson_mutex`	RW	267	holds	yes	none	**no**
`rw_readers_pref`	RWR	1,797	holds	yes	**P1(W)**	yes
`rw_writers_pref`	RWR	6,876	holds	yes	none	yes
`rw_writers_pref_2w`	RWW	5,210	holds	yes	**P0(R)**	**no**
`rw_fair`	RWR	2,715	holds	yes	none	yes
`rw_barrier`	RWR	6,876	holds	yes	none	yes
`rw_monitor_mesa`	RWR	3,512	holds	yes	**all of them**	yes
`rw_monitor_fifo`	RWR	2,756	holds	yes	**P1(W)**	yes
`rw_dekker`	RW	155	holds	yes	none	**no**
`rw_dekker_2readers`	RR	2,430	holds	yes	none	yes
`rw_peterson`	RW	116	holds	yes	none	**no**
`rw_peterson_2readers`	RR	1,382	holds	yes	none	yes
`rw_broken_count`	RWR	144	**VIOLATED**	**no**	none	**no**

Table 1. Exhaustive model checking, 2 to 3 processes per solution

Exhaustive model checking: every verdict, 2-3 processes

solution	roles	states	safe	deadlock free	starves	concurrent reads
sem mutex	RW	28	True	True	all	False
monitor mesa	RW	146	True	True	none	False
monitor fifo	RW	122	True	True	all	False
dekker	RW	443	True	True	all	False
peterson	RW	267	True	True	all	False
readers pref	RWR	1797	True	True	P1	True
writers pref (1w)	RWR	6876	True	True	all	True
writers pref (2w)	RWW	5210	True	True	P0	False
fair fifo	RWR	2715	True	True	all	True
writer barrier	RWR	6876	True	True	all	True
monitor mesa	RWR	3512	True	True	none	True
monitor fifo	RWR	2756	True	True	P1	True
dekker rw	RW	155	True	True	all	False
dekker rw (2r)	RR	2430	True	True	all	True
peterson rw	RW	116	True	True	all	False
peterson rw (2r)	RR	1382	True	True	all	True
broken (off by one)	RWR	144	False	False	all	False

Figure 2. The same verdicts as a picture: green holds, red does not, orange starves somebody

The two rows that carry the argument are the Mesa monitor and the monitor with a fair entry queue. They are the same program apart from two instructions, and one of them starves every participant while the other starves only the writer. Everything else in the table is either a mutex doing what a mutex does, or a readers-writers solution paying for fairness in one currency or another.

4.1 Reading the table

Every mutual exclusion solution passes two properties and fails the third

The semaphore mutex, the monitor mutex and both classic two-process algorithms are safe and deadlock free, and none of them ever lets two readers in at once. They satisfy the letter of the problem and miss its point entirely. The last column is the one the problem exists for.

Readers preference starves the writer; writers preference needs two writers to starve the reader

`rw_readers_pref` is safe, deadlock free and admits concurrent reads, and the writer never gets in: two readers can keep the count above zero for ever by each decrementing and re-incrementing before the other reaches zero. The mirror image is asymmetric in an informative way. With a single writer (`rw_writers_pref`), nothing starves: the reader door reopens every time the writer is done. Add a second writer (`rw_writers_pref_2w`) and the door never reopens, and the reader is the one that starves. The starvation is a property of the *load*, not only of the algorithm, and a model with one writer cannot see it.

A Mesa monitor does not bound waiting; a fair entry queue fixes it

`rw_monitor_mesa` is safe and deadlock free and admits concurrent reads, and every participant can starve. The reason is the one detail Mesa semantics leave open: a notified thread has to re-acquire the monitor lock, and a newcomer can take it first. That is how Java behaves, and it is why `notify` carries no fairness guarantee. Change one thing — hand the woken thread a place in the entry queue instead of making it race — and `rw_monitor_fifo` becomes exactly as fair as readers preference: the writer can starve, nobody else can.

> **Note:** This is the clearest result in the table, because the two monitor models differ in one instruction. What was missing was never the condition variable. It was the entry queue.

4.2 How big each proof was

The state spaces differ by more than two orders of magnitude, and the size is not a measure of how complicated the protocol is — it is a measure of how much freedom the interleaving has. The barrier solution and writers preference explore 6,876 states; the semaphore mutex needs 28. One broken variant that was tried first, a lost update on the reader count, passed 800,000 states without finishing and was replaced by a cheaper mistake: an off-by-one in the guard, which collapses in 144 states.

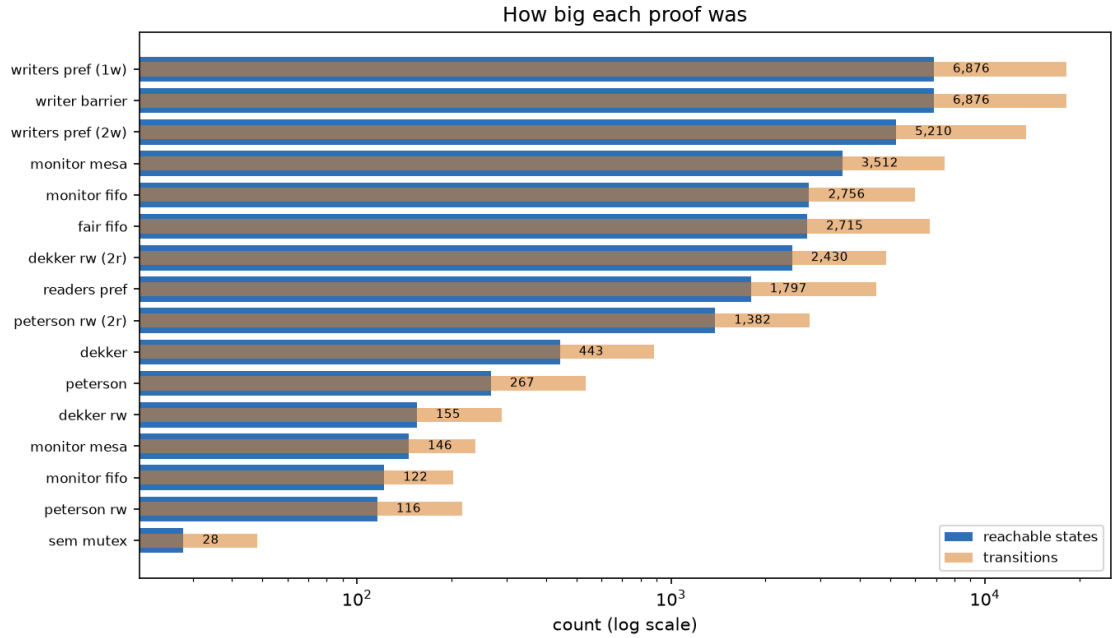


Figure 3. Reachable states and transitions per solution, log scale

Note: A broken solution is worth modelling on purpose. It is the only way to know that the checker is looking: a checker that has never failed a solution has not been tested.

4.3 What a counterexample looks like

A verdict without a witness is an assertion. The checker prints the shortest interleaving that breaks the property, and two of them are worth reading in full.

Safety, broken by one comparison

`rw_broken_count` is readers preference with the guard off by one: the first reader tests whether the count is zero instead of whether it is one. It is one comparison away from a correct solution, and the checker reaches a writer inside together with a reader in 144 states:

```
pcs=(0, 0, 0) {mutex=1q[], wrt=1q[]} shared=(0,)
P0(R): pcs=(1, 0, 0) {mutex=0q[], wrt=1q[]} shared=(0,)
P0(R): pcs=(2, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(3, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(5, 0, 0) {mutex=0q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(6, 0, 0) {mutex=1q[], wrt=1q[]} shared=(1,)
P0(R): pcs=(7, 0, 0) {mutex=1q[], wrt=1q[]} shared=(1,) ENTER
P1(W): pcs=(7, 1, 0) {mutex=1q[], wrt=0q[]} shared=(1,)
P1(W): pcs=(7, 2, 0) {mutex=1q[], wrt=0q[]} shared=(1,) ENTER
```

The safety witness for `rw_broken_count`

P0 raises the count to one, skips `wrt` because the guard tests the wrong value, and enters. P1 then takes `wrt` — which is free, because nobody ever took it — and enters too. Two lines of the trace, and the invariant is gone.

4.4 Starvation, found as a fair cycle

The Mesa monitor starves every participant, and this is the cycle that proves it for P0. P2 loops through its whole protocol and enters; P0 is woken, re-tests, and finds the section taken again.

```
P2(R): pcs=(3, 6, 13) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 14) {m=1q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 0) {m=1q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 1) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 2) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 4) {m=0q[], cv=2q[0, 1]} shared=(0, 0)
P2(R): pcs=(3, 6, 5) {m=0q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 6) {m=1q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 7) {m=1q[], cv=2q[0, 1]} shared=(1, 0) ENTER
P2(R): pcs=(3, 6, 8) {m=1q[], cv=2q[0, 1]} shared=(1, 0)
P2(R): pcs=(3, 6, 9) {m=0q[], cv=2q[0, 1]} shared=(1, 0)
```

The starvation witness for `rw_monitor_mesa`, P0

Every process on this cycle is either scheduled or blocked, so the schedule is fair: this is not a case of the victim being ignored. P0 is notified, wakes, and loses the race for the monitor lock to a thread that arrived after it. Repeating the cycle keeps it waiting for ever, which is what “no bounded waiting” means.

> **Note:** The same cycle does not exist in `rw_monitor_fifo`, because there the notified thread is handed a place in the entry queue at signal time. One instruction, and the cycle disappears.

5. Dekker and Peterson: correct, fast, and only two of you

As locks, they are correct

Both algorithms are modelled with their own instruction extensions — a process needs to know its own index to set its own flag and read the other's — and both come out safe, deadlock free and starvation free. Dekker explores 443 states and Peterson 267. Peterson is the shorter and the cheaper proof, which matches its reputation.

They can carry the pattern, between two participants

The interesting question is not whether Dekker and Peterson work as mutexes — they do — but whether the readers-writers pattern can be built on top of them. It can. Put the lock around the *reader count* instead of around the section, let the first reader take the writer semaphore, and two readers end up inside together. `rw_dekker_2readers` and `rw_peterson_2readers` are both safe, deadlock free, starvation free, and admit concurrent reads with no semaphore anywhere in the model.

solution	roles	states	safety	concurrent reads
`rw_dekker`	RW	155	holds	no
`rw_dekker_2readers`	RR	2,430	holds	yes
`rw_peterson`	RW	116	holds	no
`rw_peterson_2readers`	RR	1,382	holds	yes

Table 4. The classic algorithms carrying the readers pattern

And that is where they stop

Both algorithms are written for exactly two processes. There is no third flag and no third turn. Applied to a readers-writers problem with more than two participants they collapse into a mutex: one lock around everything, one at a time, and the concurrency the problem exists for is gone. That is not a criticism of the algorithms — it is the definition of what they are for — but it is the answer to the comparison this exercise asks for. The semaphore and the monitor express the problem for any number of participants. Dekker and Peterson express mutual exclusion for two.

> **Note:** The measurement says the same thing from the other side: run with one reader and one writer, `dekker_rw` and `peterson_rw` split the traffic almost evenly, with write shares of 0.42 and 0.45. There is no third participant for them to be unfair to.

6. What the locks cost

Four readers and two writers, an empty critical section, 5 runs per lock and the median reported with the spread beside it. Read this table as a ranking of *overhead*, not of parallelism: CPython's threads do not run in parallel, so no lock here gets faster by adding readers.

lock	ops/s (median)	spread	write share	most readers inside
semaphore mutex	37,389	25,678	0.228	1
semaphore mutex, no queue	552,192	76,847	0.392	1
monitor mutex	875,178	69,875	0.329	1
Dekker	503,579	89,154	0.497	1
Peterson	32,648	39,935	0.003	1
readers preference	28,345	10,301	0.000	1
readers preference, no queue	405,950	45,487	0.524	3
writers preference	24,850	8,598	0.193	2
fair FIFO	27,813	5,278	0.226	2
fair FIFO, no queue	317,589	38,001	0.638	3
writer barrier	22,931	13,590	0.199	1
Mesa monitor	1,307,001	160,850	0.000	4
monitor, FIFO entry	990,817	395,166	0.275	4
Dekker as count lock	64,371	15,558	0.435	1
Peterson as count lock	62,853	16,929	0.379	1

Table 5. Throughput, 4 readers + 2 writers, empty section

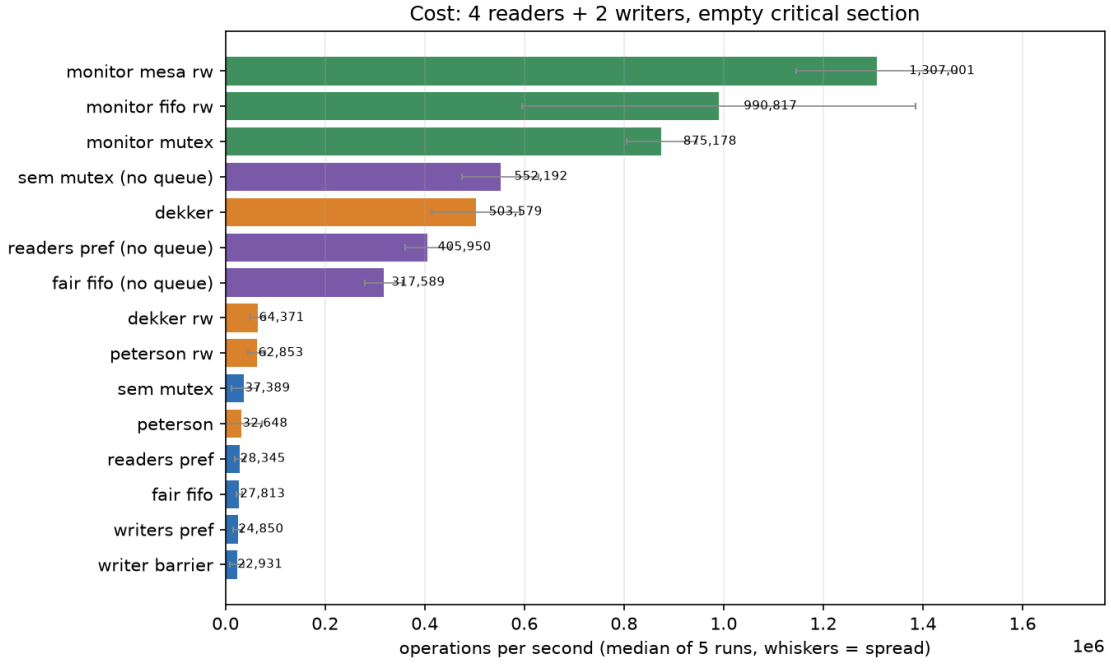


Figure 6. Throughput per lock; whiskers are the spread over repeated runs

6.1 What the no-barging queue costs

The fairness proofs in section 4 assume a semaphore that does not let a newcomer overtake a waiter. That property is not free, and this is the bill.

solution	strict FIFO semaphore	threading.Semaphore	the queue costs
semaphore mutex	37,389	552,192	14.8x
readers preference	28,345	405,950	14.3x
fair FIFO	27,813	317,589	11.4x

Table 7. The surcharge for the property the proofs assume

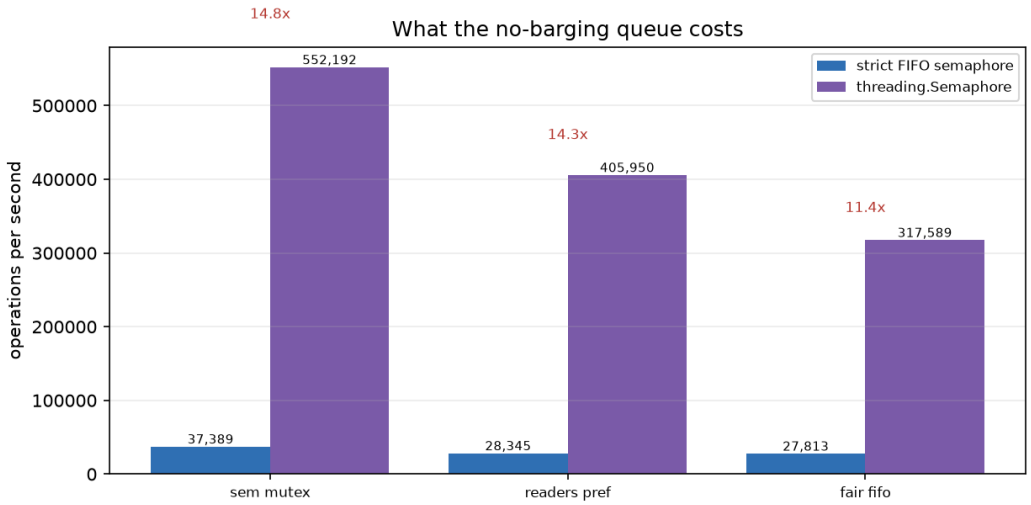


Figure 8. Same algorithm, same load, two semaphore implementations

Note: This is the honest reading of the throughput table: the gap between the monitor solutions and the semaphore solutions is mostly the cost of the FIFO queue in this implementation, not a property of the algorithms. Compare within a family, not across one.

7. The property the problem exists for

Six readers, one writer, and real work inside the section so that overlap is possible at all. The number is the largest count of readers observed inside simultaneously. A plain mutex cannot score more than one; anything that scores one is a mutex wearing a label.

lock	most readers inside at once	write share
fair FIFO, no queue	6	0.785
monitor, FIFO entry	6	0.000
Mesa monitor	6	0.000
readers preference	6	0.716
readers preference, no queue	6	0.000
fair FIFO	5	0.104
writers preference	4	0.105
writer barrier	3	0.127
monitor mutex	1	0.000
semaphore mutex	1	0.137
semaphore mutex, no queue	1	0.000

Table 9. Concurrent readers, 6 readers + 1 writer, work inside the section

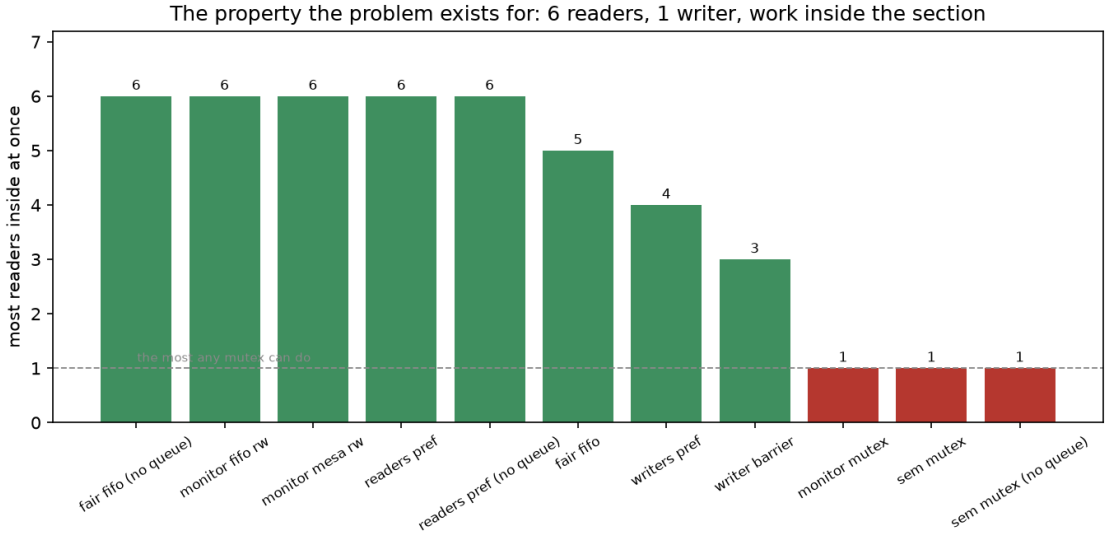


Figure 10. Green reaches two or more readers, red never does

Note: The barrier solution caps overlap on purpose — that is what a barrier is for — so a lower number there is the design working, not the design failing. It reaches 3 here against 6 for readers preference.

8. Starvation, measured

Six readers against one writer

This is the load readers preference is supposed to fail, and it does. The figure records every trial separately rather than averaging them, because several of these locks are bimodal: the outcome depends on who wins the opening race and then stays that way for the rest of the trial. A median of two modes is a number that describes neither.

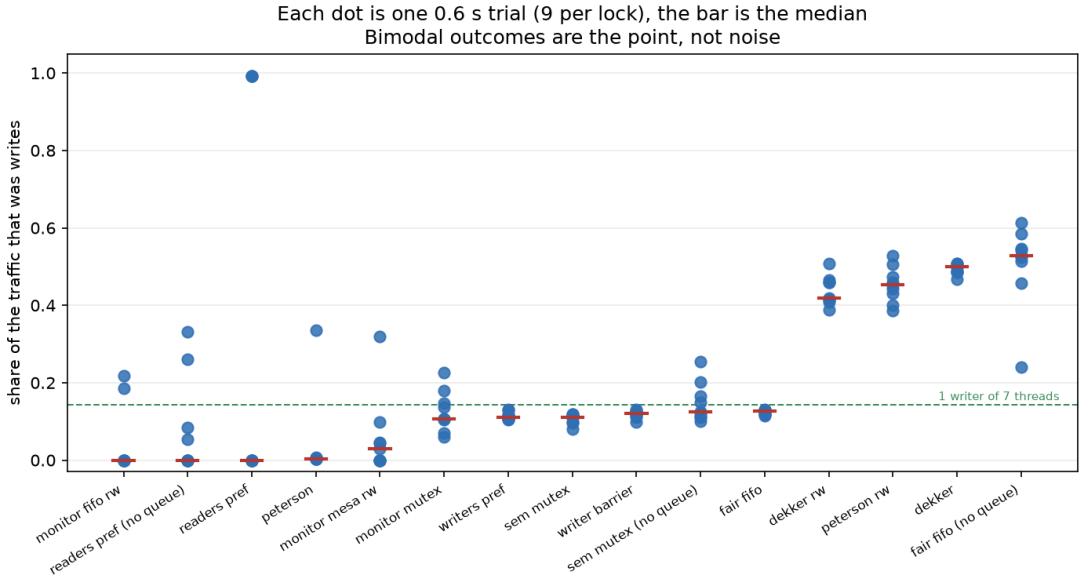


Figure 11. Each dot is one 0.6 s trial, the red bar is the median, the green line is 1 writer of 7 threads

8. Starvation, the numbers

lock	write share (median)	trials in which the writer starved	longest unbroken read run	longest unbroken write run
monitor, FIFO entry	0.000	7/9	1,094,802	55,557
readers preference, no queue	0.000	5/9	187,707	38,426
readers preference	0.000	6/9	10,895	19,850
Peterson	0.004	8/9	16,752	6,465
Mesa monitor	0.030	4/9	1,252,307	33,857
monitor mutex	0.106	0/9	177,626	40,395
writers preference	0.111	0/9	3,458	1
semaphore mutex	0.111	0/9	10,608	1
writer barrier	0.121	0/9	4,742	1
semaphore mutex, no queue	0.124	0/9	107,483	20,546
fair FIFO	0.128	0/9	2,222	1
Dekker as count lock	0.418	0/9	9,282	3,214
Peterson as count lock	0.454	0/9	8,098	5,421
Dekker	0.499	0/9	16,039	14,050
fair FIFO, no queue	0.528	0/9	36,382	30,729

Table 12. 9 trials per lock, 6 readers + 1 writer

8.1 Reading the starvation figures

Readers preference, and both monitors, starve the writer

Readers preference starved the writer in 6 of 9 trials; the Mesa monitor did it in 4 and the FIFO-entry monitor in 7. The longest unbroken read run for readers preference is 10,895 operations — a read phase that never ended. The fair FIFO turnstile, the writer barrier, writers preference and both mutexes never starved anybody in any trial.

Peterson's spinlock is the surprise

Peterson's algorithm starved the writer in 8 of 9 trials, and Dekker's was writer-dominant in 3 of them. Both are provably starvation free, and the model checker says so. The measurement disagrees for a reason that has nothing to do with the algorithms: a spinning thread holds the CPU and does nothing with it. Under a global interpreter lock the spinning thread burns its whole quantum, and the other thread gets thousands of operations done in its own. The spin counts make it plain — the two-process locks recorded 17,548,790 spins in a quarter of a second.

> **Note:** This is the clearest case in the exercise of a proof and a measurement disagreeing without either being wrong. The proof is about fair schedules. The measurement is about CPython's scheduler, which is not obliged to be fair to a thread that is spinning.

8.2 The mirror image, and where measurement gives up

One reader against four writers. Writers preference keeps the reader door shut while any writer is waiting, so the reader's share should collapse.

lock	write share (median)	longest unbroken write run
fair FIFO, no queue	0.959	122,344
readers preference, no queue	0.883	90,717
writers preference	0.845	19,135
monitor, FIFO entry	0.806	319,199
writer barrier	0.802	21,323
monitor mutex	0.748	124,640
Mesa monitor	0.719	150,744
readers preference	0.700	3,238
semaphore mutex, no queue	0.678	78,106
fair FIFO	0.666	2,484
semaphore mutex	0.589	3,340

Table 13. 1 reader against 4 writers

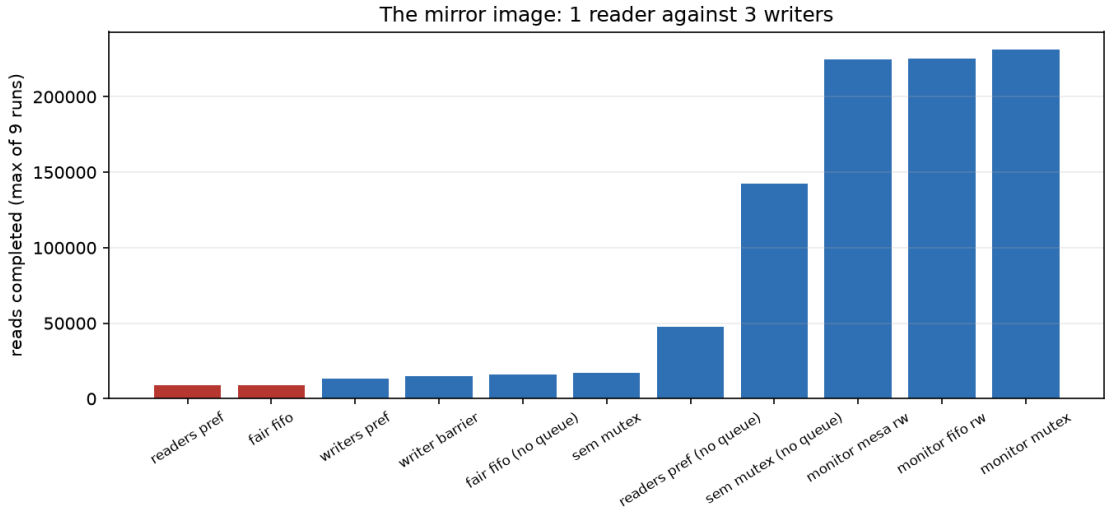


Figure 14. Reads completed under a writer-heavy load

What the measurement can and cannot show here

It cannot show what the proof shows. Writers preference does produce long unbroken write runs where the FIFO solutions alternate every few operations, but its write share is not reliably distinguishable from the baselines, and Python's own monitor lock is at this timescale *less* fair than writers preference is. The share of the traffic over one second is dominated by who won the opening race, and by the fairness of the interpreter's own lock, not by the algorithm. The honest statement is the one the model checker supplies: with two writers, writers preference starves the reader for ever. The measurement corroborates the mechanism and cannot isolate the effect.

> **Note:** A measurement that cannot separate signal from noise is still worth reporting, as long as it is reported as such. The temptation is to quote the one run that shows what the theory predicts.

9. Does adding readers help?

One writer, one to eight readers, work inside the section. On a machine with real parallelism this is where a readers-writers lock should pull away from a mutex. On CPython it does not, because the threads never run at the same time: what the graph shows is lock overhead, and the honest conclusion is that none of these solutions scales with the number of readers here.

lock	1 reader	2	4	6	8
Mesa monitor	100,105	103,152	91,676	98,488	102,698
monitor, FIFO entry	95,914	96,835	93,861	98,692	93,208
fair FIFO	19,149	23,452	19,566	21,505	19,815
writer barrier	21,492	22,123	23,269	19,106	17,588
semaphore mutex	26,988	22,933	21,235	17,144	17,303
writers preference	20,182	21,668	20,683	15,166	14,562
readers preference	25,146	91,155	73,629	13,266	12,478

Table 15. Operations per second against the number of reader threads, plus one writer

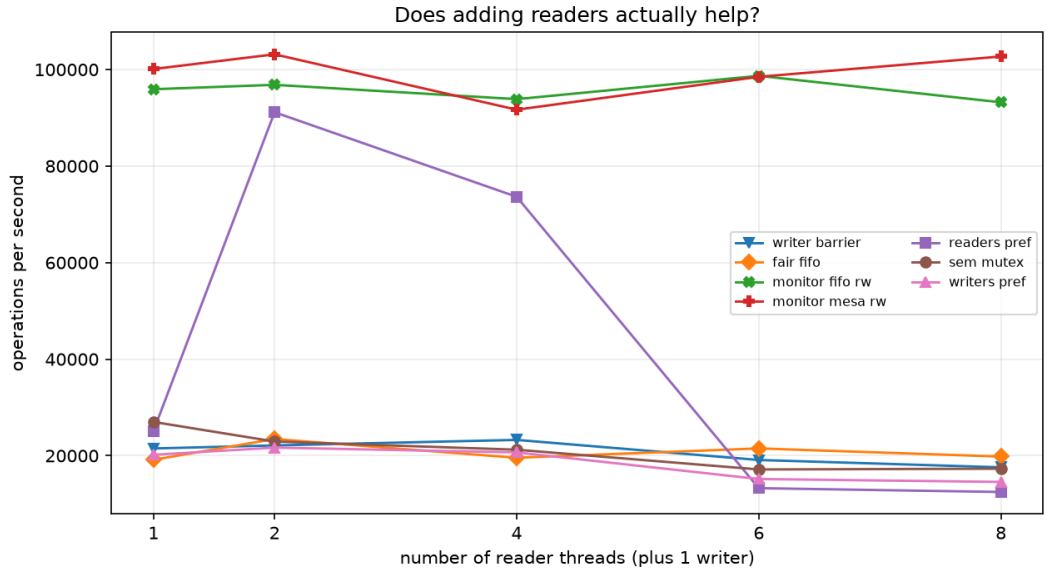


Figure 16. Throughput against reader count

Note: The one real movement in the table is readers preference going from 25,146 ops/s with one reader to 91,155 with two — the moment two readers can overlap. It does not continue, because the bottleneck is the interpreter, not the lock.

10. Which one to use

If you need	Use	Because
Any number of participants, no fairness guarantee needed	readers preference	fastest of the semaphore family and the simplest; the writer can starve
Nobody to starve	fair FIFO turnstile	the only semaphore solution here with no starvation in any trial; the writer holds a semaphore across its whole wait
Bounded phases	writer barrier	two symmetric gates; caps reader overlap on purpose so writers are not locked out
A monitor	monitor with a fair entry queue	Mesa semantics alone do not bound waiting; the entry queue is not an optional refinement
Exactly two participants, no OS support	Peterson	shorter and cheaper to prove than Dekker, and both are correct and starvation free
More than two participants	not Dekker, not Peterson	both are written for two; beyond that they degrade to a mutex

Table 17. What the verdicts support

The one lesson that generalises

Fairness is not a property of the primitive, it is a property of the queue. A semaphore has one and you can rely on it. A monitor does not, and the difference between a monitor that starves everybody and one that starves only the writer is a single instruction about where a woken thread waits.

11. Method, artefacts and limits

What is on this branch

File	What it is
`code/rw_model.py`	the exhaustive model checker, 17 protocols
`code/rw_sync.py`	the threaded implementations and the benchmark harness
`code/run_experiments.py`	runs every experiment, writes `output/experiments.json`
`code/make_diagrams.py`	regenerates every figure
`tests/test_exercise7.py`	102 tests
`output/experiments.json`	every number in this paper
`diagrams/`	9 figures, all regenerable
`docs/REPORT_EN.docx / .pdf`	this paper, English
`docs/REPORT_FA.docx / .pdf`	this paper, Persian

Table 18. Files published on the Exercise-7 branch

Limits, stated plainly

- The model checker covers two or three processes. It proves properties of the protocol, not of any number of threads.
- The measurements run on CPython, where threads do not execute in parallel; throughput figures are overhead, not scalability.
- The strict FIFO semaphore in this exercise is a deliberately simple implementation, and it is roughly an order of magnitude slower than `threading.Semaphore`. Section 5.1 prices that.
- Starvation results at the half-second scale are bimodal for several locks; every trial is reported rather than averaged.
- Nothing here is sampled. The model checking is exhaustive and the harness checks the invariant on every entry.

Exercise	7 — readers and writers
Branch	Exercise-7-Readers-Writers-Comparison
Solutions modelled	17
Locks benchmarked	15
Tests passing	102
Python	3.11.2

Table 19. At a glance